1. Write a C program to simulate a Deterministic Finite Automata (DFA) for the given language representing strings that start with a and end with a

Aim:

To write a C program to simulate a Deterministic Finite Automaton (DFA) that accepts strings starting with a and ending with a over the alphabet $\Sigma = \{a, b\}$.

Algorithm:

1. Start the program.
2. Read the input string containing a and b.
3. Check whether the first character is a.
4. Check whether the last character is a.
5. Verify that all characters belong to the alphabet {a, b}.
6. If all conditions are satisfied, display "String is ACCEPTED"; otherwise, display "String is REJECTED".
7. Stop the program.

Program :

```c
#include <stdio.h>
#include <string.h>
int main()
{
    char str[100];
    int state = 0, i;
    printf("Enter a string containing a and b: ");
    scanf("%99s", str);
    for (i = 0; i < strlen(str); i++)
    {
        if (state == 0)
        {
            if (str[i] == 'a')
                state = 1;
            else
                state = 3;
        }
        else if (state == 1)
        {
            if (str[i] == 'a')
                state = 2;
            else
```

main.c

```
                state = 2;
            else
                state = 1;
        }
        else if (state == 2)
        {
            if (str[i] == 'a')
                state = 2;
            else
                state = 1;
        }
        else
            state = 3;
    }

    if (state == 2)
        printf("String is ACCEPTED");
    else
        printf("String is REJECTED");

    return 0;
}
```

Output :

```
Enter a string containing a and b: aaaaba
String is ACCEPTED

...Program finished with exit code 0
Press ENTER to exit console.
```

Result :

Thus, the C program to simulate a DFA for strings that start with a and end with a was successfully executed and the expected output was obtained.

2. Write a C program to simulate a Non-Deterministic Finite Automata (NFA) for the given language representing strings that start with o and end with 1

Aim:

To write a C program to simulate a Non-Deterministic Finite Automaton (NFA) that accepts strings starting with 0 and ending with 1 over the alphabet $\Sigma = \{0, 1\}$.

Algorithm:

1. Start the program.
2. Read the input string containing `0` and `1`.
3. Check whether the first character is `0`.
4. Check whether the last character is `1`.
5. Verify that all characters belong to the alphabet `{0, 1}`.
6. If all conditions are satisfied, display "String is ACCEPTED"; otherwise, display "String is REJECTED".
7. Stop the program.

Program:

main.c

```c
#include <stdio.h>
#include <string.h>

int main()
{
    char str[100];
    int i, n, valid, j;

    for (j = 1; j <= 2; j++)
    {
        printf("Enter binary string %d: ", j);
        scanf("%99s", str);

        n = strlen(str);
        valid = 1;

        if (str[0] != '0' || str[n - 1] != '1')
        {
            valid = 0;
        }

        for (i = 0; i < n; i++)
```

```
        {
            if (str[i] != '0' && str[i] != '1')
            {
                valid = 0;
                break;
            }
        }

        if (valid)
        {
            printf("String is ACCEPTED\n");
        }
        else
        {
            printf("String is REJECTED\n");
        }
    }

    return 0;
}
```

output:

```
Enter binary string 1: 001
String is ACCEPTED
Enter binary string 2: 1100
String is REJECTED
```

Result:

Thus, the C program to simulate an NFA for strings that start with 0 and end with 1 was successfully executed, and the accepted and rejected strings were displayed correctly.

3. Write a C program to find ε -closure for all the states in a Non-Deterministic Finite Automata (NFA) with ε -moves

Aim:

To write a C program to find the ε-closure of all states in a Non-Deterministic Finite Automaton (NFA) with ε-moves.

Algorithm:

1. Start the program.

2. Read the number of states and the number of ε-transitions.
3. Read the ε-transitions between the states.
4. Initialize the ε-closure of each state with the state itself.
5. Find all states reachable through ε-transitions from each state.
6. Repeat until no new states can be added to the closure.
7. Display the ε-closure of every state.
8. Stop the program.

Program:

main.c

```c

#include <stdio.h>

int main()
{
    int n, e, i, j, k;
    int closure[20][20] = {0};
    int from, to, changed;

    printf("Enter number of states: ");
    scanf("%d", &n);

    printf("Enter number of epsilon transitions: ");
    scanf("%d", &e);

    for (i = 0; i < n; i++)
        closure[i][i] = 1;

    printf("Enter epsilon transitions (from to):\n");

    for (i = 0; i < e; i++)
    {
```

main.c

```
23          scanf("%d %d", &from, &to);
24
25          if (from >= 0 && from < n && to >= 0 && to < n)
26              closure[from][to] = 1;
27      }
28
29      do
30      {
31          changed = 0;
32
33          for (i = 0; i < n; i++)
34          {
35              for (j = 0; j < n; j++)
36              {
37                  if (closure[i][j])
38                  {
39                      for (k = 0; k < n; k++)
40                      {
41                          if (closure[j][k] && !closure[i][k])
42                          {
43                              closure[i][k] = 1;
44                              changed = 1;
45                          }
```

```c
hain.c
                }
            }
        }
    } while (changed);

    printf("\nEpsilon-Closure of all states:\n");

    for (i = 0; i < n; i++)
    {
        printf("E-closure(q%d) = { ", i);

        for (j = 0; j < n; j++)
        {
            if (closure[i][j])
                printf("q%d ", j);
        }

        printf("}\n");
    }

    return 0;
}
```

Output:

```
Enter number of states: 3
Enter number of epsilon transitions: 2
Enter epsilon transitions (from to):
0 1
1 2

Epsilon-Closure of all states:
E-closure(q0) = { q0 q1 q2 }
E-closure(q1) = { q1 q2 }
E-closure(q2) = { q2 }
```

Result:

Thus, the C program to calculate the ε-closure of all states in an NFA with ε-transitions was successfully executed, and the ε-closure of each state was displayed.